

Task 5: AI-Powered Real-World Application

Submitted By:	G. Pavana Deepika
Task ID:	Task 5 - AI-Powered Real-World Application
Tech Stack:	Python, Streamlit, Scikit-Learn, Pandas & NumPy
Target Domain:	End-to-End Predictive Web Application & Model Inference

1. Introduction & Project Overview

This project implements a complete, end-to-end **AI-Powered Real-World Application** designed to solve practical predictive problems. By integrating machine learning models with an interactive user interface, the application bridges raw data processing with dynamic feature input, executing real-time inference and generating data-driven predictions.

2. Project Objectives & Requirements

The primary objectives of Task 5 are fully satisfied through the following core components:

- **AI/ML Component:** Integration of a trained predictive model pipeline for real-time data inference.
- **User Input Interface:** Dynamic UI elements allowing users to input custom configuration parameters and attributes.
- **Data & Model Processing:** Automated preprocessing, feature scaling, and inference execution.
- **Meaningful Prediction Output:** Instantaneous, data-driven regression or classification results displayed cleanly to the user.

3. Technology Stack & Libraries

- **Python:** Core programming language utilized for application logic and model orchestration.
- **Streamlit:** Framework used for developing and deploying interactive web-based user interfaces.
- **Scikit-Learn / Pandas:** Used for building data processing pipelines, feature matrix transformations, and numerical computations.

4. Pipeline & Implementation Methodology

The application architecture follows a structured end-to-end workflow:

- **Input Collection Layer:** Captures user parameters via interactive sliders and select boxes through the Streamlit sidebar.
- **Feature Processing Pipeline:** Formats raw inputs into structured NumPy arrays suitable for model ingestion.
- **Inference & Scoring Engine:** Computes weighted regression formulas to generate real-time predictive outputs.
- **Presentation Layer:** Renders visual metrics, status alerts, and structured data frames dynamically.

5. System Design & Feature Workflow

Component Layer	Technology / Tool	Functional Description
User Interface	Streamlit Web App	Provides clean, responsive sidebar inputs and metric displays.
Data Processing	Pandas & NumPy	Transforms raw user inputs into normalized feature matrices.
Model Inference	Scikit-Learn / Custom Logic	Evaluates processed features to return robust predictive metrics.

6. Step-by-Step Execution Guide

To run and evaluate the application locally:

```
# 1. Install required dependencies
pip install streamlit pandas scikit-learn numpy

# 2. Save script as app.py inside your task folder
# 3. Launch application via terminal
streamlit run app.py
```

7. Deliverables & Verification

The application is fully complete with a structured GitHub repository, functional python application script, interactive user interface, complete technical documentation, and execution guides.

Status: Completed & Verified